UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
Centro de Ciências Exatas e da Terra - Departamento de Informática e Matemática Aplicada

DIM0411 - Image Processing - Prof. Bruno Motta de Carvalho

Image Compression Using the Discrete Cosine Transform (DCT)

Transform-Based (Lossy) Image Compression Project

Eduardo Marinho
Kael Paraguassu

July 09, 2026

1 Introduction

This project implements a lossy, transform-based image compression pipeline based on the same block-wise Discrete Cosine Transform (DCT) scheme used by the JPEG standard. A grayscale image is divided into 8×8 pixel blocks, each block is transformed into the frequency domain, its coefficients are quantized (which discards information and introduces sparsity), and the image is then reconstructed through inverse quantization and inverse DCT. The trade-off between reconstruction quality and compression is controlled by a user-defined compression factor that scales a standard JPEG luminance quantization matrix.

The program was implemented in C++ using OpenCV strictly for the image I/O and matrix display, all DCT, quantization, and metric computations were implemented from scratch. Quality is evaluated with the Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR), and the compression rate (CR) is estimated from the number of coefficients that survive quantization.

2 Methodology

2.1 Pipeline Overview

The image is read in grayscale and zero-padded so that both dimensions become multiples of 8. The padded image is then processed independently, block by block (8×8 pixels), through the following stages:

- **Forward DCT** → convert the pixel block to the frequency domain.
- **Quantization** → divide each frequency coefficient by a (scaled) quantization step and round to the nearest integer, discarding small/high-frequency components.
- **Inverse quantization** → multiply the quantized coefficients back by the same quantization step.

- **Inverse DCT** → convert the dequantized coefficients back to the spatial (pixel) domain.

Reconstructed blocks are reassembled into the final image, and the original padding is cropped away before saving the output and computing quality metrics.

2.2 Discrete Cosine Transform

Before transforming, each pixel value is level-shifted by subtracting 128 so that values are centered around zero. The 2D forward DCT of an 8×8 block $f(x, y)$ is computed as:

$$F(u,v) = \frac{1}{4}\, C(u)\, C(v) \sum_{x=0}^{7} \sum_{y=0}^{7} \big[f(x,y) - 128\big] \cos\left[\frac{(2x+1)u\pi}{16}\right] \cos\left[\frac{(2y+1)v\pi}{16}\right] \tag{1}$$

where $x, y, u, v \in \{0, \ldots, 7\}$, and

$$C(k) = \begin{cases} 1/\sqrt{2}, & k = 0 \\ 1, & k \neq 0 \end{cases} \tag{2}$$

The inverse DCT applies the same cosine basis in reverse, adds 128 back to undo the level shift, and clips the result to the valid $[0, 255]$ range before rounding to an integer pixel value.

2.3 Quantization

Quantization uses the standard JPEG luminance quantization matrix Q (8×8), scaled by the user defined compression factor f: each step is

$$Q'(i,j) = \max\big(1,\ Q(i,j) \cdot f\big). \tag{3}$$

Each DCT coefficient is divided by its corresponding step and rounded:

$$F_q(u,v) = \text{round}\left[\frac{F(u,v)}{Q'(i,j)}\right]. \tag{4}$$

Larger values of f enlarge the quantization steps, driving more coefficients, especially the high-frequency ones, to zero. This is what produces compression, the fraction of nonzero coefficients that remain after this step directly determines the achievable compression rate. Smaller values of f preserve more coefficients and produce reconstructed images that more closely resemble the original.

2.4 Reconstruction

To reconstruct an approximation of the original block, each quantized coefficient is multiplied back by the same step used to quantize it (dequantization: $\hat{F}(u,v) = F_q(u,v) \cdot Q'(i,j)$), and the inverse DCT described in Section 2.2 is applied to recover the pixel block.

2.5 Evaluation Metrics

Reconstruction error is measured with the Mean Squared Error between the original image I and the reconstructed image K (both $m \times n$):

$$\text{MSE} = \frac{1}{m \cdot n} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} \big[I(i,j) - K(i,j)\big]^2 \tag{5}$$

Quality is then reported as the Peak Signal-to-Noise Ratio, in decibels, using $\text{Max}_I = 255$ for an 8-bit image:

$$\mathrm{PSNR} = 20 \cdot \log_{10}\left(\frac{\mathrm{Max}_I}{\sqrt{\mathrm{MSE}}}\right) \tag{6}$$

The compression rate is estimated as the ratio between the uncompressed size (total number of pixels) and the number of nonzero quantized coefficients retained across all blocks. This isolates and measures the sparsification effect of quantization, it does not include the additional gain that a final entropy-coding stage (e.g. Huffman or run-length coding, as in real JPEG) would provide on top of it.

3 Results

The program was used to compress a sample grayscale test image using four compression factors, ranging from a light setting (1.0) to a heavy setting (8.0), to illustrate the quality/compression trade-off described in Section 2.3.

3.1 Input Image

Figure 1: Original grayscale input image.

3.2 Output Images at Different Compression Levels

(a) Compression factor = 1

(b) Compression factor = 2

(c) Compression factor = 4

(d) Compression factor = 8

Figure 2: Reconstructed image at compression factors 1.0, 2.0, 4.0, and 8.0, respectively.

3.3 Quantitative Results

Compression Factor	MSE	PSNR (dB)	Compression Ratio
1.0	0.00965	68.285	9.14732x
2.0	24.7137	34.2014	12.2669x
4.0	44.9737	31.6012	21.9221x
8.0	88.5617	28.6583	35.3627x

The results confirm the expected quality/compression trade-off, however, they reveal to have a non-linear compression rate and data loss. At factor 1.0, reconstruction results highly resemble

the original image, since the quantization steps are close to the base JPEG matrix and almost every DCT coefficient survives. Intuitively, this is also the setting with the lowest compression ratio (9.15×). Raising the factor to 2.0 causes a disproportionate drop in quality, as PSNR drops by over 34 dB (68.285 → 34.201 dB), for a comparatively small gain in compression ratio (9.15× → 12.27×). Beyond that point the trade-off becomes more favorable: going from factor 2.0 to 8.0 nearly triples the compression ratio (12.27× → 35.36×) while PSNR falls by only about 5.5 dB (34.201 → 28.658 dB), with MSE increasing more gradually (24.71 → 88.56) than in the initial jump.

This pattern indicates that most of the perceptible quality loss occurs when moving from low to moderate compression, as many mid- and high-frequency coefficients are first rounded to zero. After that, increasing the compression factor mainly removes coefficients with less significant information, resulting in smaller PSNR reductions. Visually, this is consistent with sharp detail and edges being preserved at factor 1.0, while 8×8 blocking artifacts and loss of fine texture become increasingly visible from factor 2.0 onward, most noticeably at factor 8.0.

The quantitative results clearly corroborate with our visual observations, since the image 'a' (compression rate = 1) is almost equal to the original image, and, as the compression factor increases, the image quality decays significantly.

4 Conclusion

The experiments confirm the fundamental trade-off inherent to DCT-based lossy compression: increasing the compression factor increases the achievable compression ratio at the cost of reconstruction fidelity, but the two do not scale together at a constant rate. The quantitative results suggests that, for this image, a factor around 2.0 already achieves most of the compression gain before the improvements in compression rate become less significant. Because of that, pushing further to factor 8.0 is only worthwhile when a much higher compression ratio is required and the accompanying blocking artifacts and loss of fine detail are acceptable.